Real-world (rebuild Project 2 as a graph) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangGraph, plus proving equivalence with your original loop, not just "it runs").

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

Your Doc07 loop already has the whole shape: ask the model → if it wants tools, run them and add the results → ask again → stop when it answers. A graph is that same shape drawn as boxes and arrows. "Ask the model" becomes one node, "run the tools" becomes another, and the `if` that decides whether to go round again becomes a conditional edge.

Don't rewrite anything Doc07 already does. Copy Doc07's Build Task files into `09_langgraph/practice/` and import `AGENT_TOOLS` and `run_tool` from `agent.py`.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

The exact pieces:

- **State:** `messages: Annotated[list, operator.add]` and `steps: Annotated[list, operator.add]` — each node returns *new* items, and LangGraph adds them to the old list (the reducer from this document's Basic exercise).
- **`think` node:** `ChatOpenAI(...).bind_tools(AGENT_TOOLS).invoke(state["messages"])` — return `{"messages": [response]}`.
- **`act` node:** loop over `state["messages"][-1].tool_calls`, call Doc07's `run_tool(call, AGENT_TOOLS)`, and return the `{"role": "tool", ...}` messages plus one step dict per call.
- **`route`:** `"act"` if the last message has `tool_calls`, otherwise `"finish"`, mapped to `END`.
- **Step limit:** `graph.invoke(start, config={"recursion_limit": 20})` — the graph's `max_iterations`.

Then ask the real question: does the graph behave the same as the loop? Answers are worded differently every run, so compare **which tools** each used, on the same prompts — and, if you want a second signal, how many model turns each took.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state: messages (added to), steps (added to)

think(state):
    reply = model with AGENT_TOOLS, invoked on state["messages"]
    return {"messages": [reply]}

act(state):
    for each call in the last message's tool_calls:
        result, error = run_tool(call, AGENT_TOOLS)
        make a {"role": "tool", ...} message, and a step dict
    return {"messages": [...], "steps": [...]}

route(state):
    return "act" when the last message has tool_calls,
    otherwise return "finish"

edges: START -> think, think -> act or END, act -> think
```

Here's almost the whole `act` node — fill in the two appends:

```
def act(state):
    tool_messages = []
    steps = []
    for call in state["messages"][-1].tool_calls:
        result, error = run_tool(call, AGENT_TOOLS)
        observation = result
        if error is not None:
            observation = "Error: " + error
        # your turn: append the tool message and the step dict
        ...
    return {"messages": tool_messages, "steps": steps}
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
build_graph() -> compiled graph          (the Basic wiring, in a function)
run_graph(graph, task) -> final state    (recursion_limit=20)

compare_with_loop(graph):
    for each test prompt:
        loop_tools  = tool names from run_agent(prompt, 5).steps   (Doc07)
        graph_tools = tool names from run_graph(graph, prompt)["steps"]
        print MATCH or MISMATCH
    raise AssertionError if any prompt mismatched

count_model_turns(messages):
    count the messages that are NOT dicts (the model's replies)
```

One piece to get you unstuck:

```
def tool_names(steps: list[dict]) -> list[str]:
    names = []
    for step in steps:
        names.append(step["tool"])
    return names
```

Pick prompts with a known right tool: one calculation, one weather question, one general-knowledge question (no tool). Then compare against the Solution.

Difference between Basic and Intermediate: Basic translates the loop into a graph that runs. Intermediate adds a step limit, puts the wiring in reusable functions, and *proves* the graph behaves like the loop instead of assuming it.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution